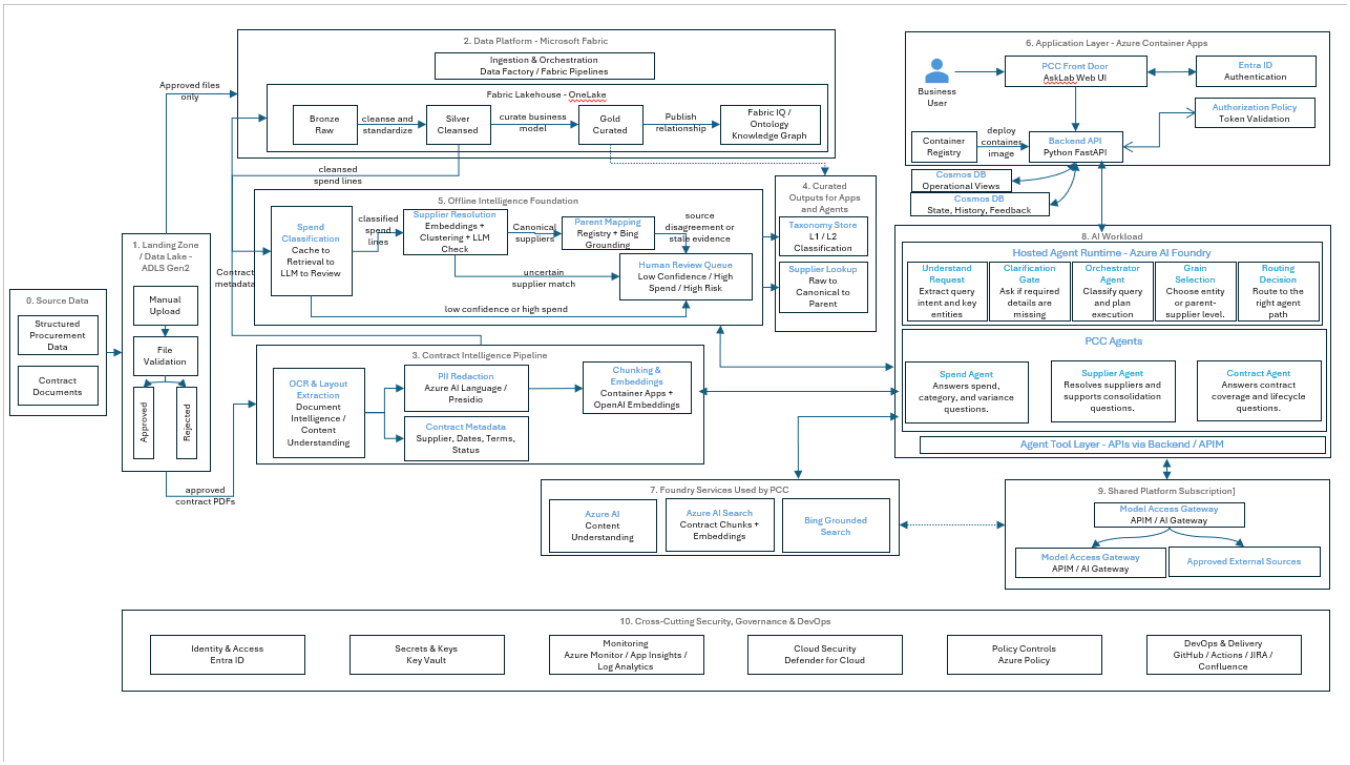
Solution - Low Level Design



1	Structured procurement files are received.	These are the core datasets for spend, PO, invoice, supplier, and taxonomy analysis.	Define accepted file formats and source file naming conventions. Capture source system, file type, load date, sensitivity classification, and data owner.	Source files, Excel/CSV extracts, ADLS Gen2 landing zone	No technical dependency. Business/source team must provide approved structured files.
2	Contract PDF files are received.	Contract data is needed to answer coverage, expiry, renewal, terms, and off-contract questions.	Accept only approved contract PDFs. Do not process unvalidated or unapproved contract documents.	PDF contracts, ADLS Gen2 landing zone	No technical dependency. Business/source team must provide approved PDFs.
3	Files are manually uploaded into the landing zone.	MVP uses file-based ingestion rather than live ERP/SAP /contract system integration.	Create separate landing folders or containers for structured files and contract PDFs. Use consistent folder structure such as /landing/structured/, /landing/contracts/, /quarantine/, /rejected/, /approved/.	ADLS Gen2	Step 1 for structured upload; Step 2 for contract upload; ADLS landing folders must exist.
4	Files first go through quarantine and validation.	Prevent unsafe, unsupported, or malformed files from entering Fabric or AI pipelines.	Implement validation before ingestion. Validate MIME type, file extension, max file size, max page count for PDFs, malware scan result, parseability, and source metadata.	ADLS Gen2, Azure Functions or Container Apps validation job, Defender/Malware scanning mechanism	Step 3; validation rules and quarantine structure must be configured.
5	Invalid files are rejected and logged.	Failed files must be traceable for audit and forensic investigation.	Move failed files to rejected/quarantine container. Write validation result, failure reason, uploader/source, timestamp, file hash, and correlation ID to a validation log table.	ADLS Gen2 rejected container, Log Analytics, Fabric control table or SQL control table	Step 4; rejected container and validation log table must exist.

6	Approved structured files are passed to Fabric ingestion.	Only validated data should enter the medallion data platform.	Trigger Fabric Pipeline or Data Factory pipeline only for approved files. Use metadata-driven ingestion where possible.	Microsoft Fabric Pipelines / Azure Data Factory, ADLS Gen2	Step 4; only approved structured files can move forward.
7	Raw data is loaded into Bronze.	Bronze preserves source-aligned raw records for traceability.	Load source files into Bronze tables with minimal transformation. Add technical metadata columns such as source_file_name, ingestion_batch_id, load_timestamp, source_system, record_hash.	Fabric Lakehouse Bronze tables, OneLake	Step 6; Fabric ingestion pipeline must be available.
8	Bronze data is transforme d into Silver.	Silver provides cleansed, standardized, deduplicated data for downstream processing.	Standardize column names, data types, dates, currencies, supplier strings, category fields, quantities, and amounts. Remove duplicates and handle nulls according to business rules.	Fabric Spark Notebooks / SQL Notebooks, Silver Lakehouse tables	Step 7; Bronze tables must be loaded successfully.
9	Data quality and reconciliati on checks run.	The system must prove that records were not lost, duplicated, or incorrectly transformed.	Implement row count checks, schema checks, null checks, duplicate checks, amount reconciliation, referential checks, and business rule checks. Store results in a DQ log.	Fabric notebooks, Fabric Data Activator/monitoring if used, Log Analytics, DQ control tables	Step 8; Silver transformation outputs must be available.
10	Validated Silver data is curated into Gold.	Gold is the trusted business-ready layer used by AskLab, Insights Lab, and agents.	Build curated fact and dimension models for PO, invoice, supplier, taxonomy, contract metadata, and relationships. Apply approved mappings and business rules.	Fabric Gold Lakehouse /Warehouse tables	Step 9; DQ checks must pass, or exceptions must be formally accepted by Data Owner/Product Owner.
11	Fabric IQ / ontology relationship s are built from Gold.	AskLab needs relationship context across supplier, PO, invoice, contract, category, region, and time.	Model entities and relationships: Supplier, Parent Supplier, PO, Invoice, Contract, Category, Region, Time Period, Buying Entity.	Microsoft Fabric IQ / ontology / knowledge graph layer	Step 10; Gold entities and relationships must be defined.
12	Contract PDFs enter OCR and layout extraction.	PDFs are unstructured and must be converted into machine-readable text and fields.	Send approved contract PDFs to Document Intelligence / AI Content Understanding. Extract layout, text, tables, clauses, and key metadata candidates.	Azure AI Document Intelligence, Azure AI Content Understanding	Step 4; only approved contract PDFs can be processed.
13	Contract text is redacted before search indexing.	Contract PDFs may contain PII, signatories, and sensitive commercial details.	Run PII detection and redaction before chunking and embedding. Only redacted approved text should go to AI Search.	Azure AI Language PII, Microsoft Presidio, custom redaction pipeline	Step 12; extracted contract text must be available.
14	Contract metadata is extracted.	Structured metadata is needed for contract lifecycle, expiry, renewal, coverage, and supplier linkage questions.	Extract fields such as supplier, parties, effective date, expiry date, renewal date, contract status, commercial terms where approved, and confidence score.	Azure AI Content Understanding, Azure Document Intelligence, Container Apps extraction service	Step 12; OCR/layout extraction must complete.
15	Contract metadata is written to Bronze, Silver, and Gold.	Contract data must follow the same governed data path as structured procurement data.	Store raw extraction JSON in Bronze, normalized contract metadata in Silver, and curated contract metadata in Gold.	Fabric Lakehouse Bronze /Silver/Gold	Step 14; metadata schema and Fabric tables must exist.
16	Redacted contract text is chunked and embedded.	Contract evidence needs to be retrievable for Contract Agent answers.	Chunk redacted text by clause/section/page. Generate embeddings. Store chunk text, metadata, contract ID, supplier ID, page /section, and access metadata.	Azure Container Apps, Azure OpenAI Embeddings, Azure AI Search	Step 13; only redacted approved text can be embedded.

17	Contract chunks are indexed in Azure AI Search.	Contract Agent needs semantic retrieval over approved redacted contract evidence.	Create AI Search index with fields such as contract_id, supplier_id, chunk_text, page_number, clause_type, effective_date, expiry_date, security_label.	Azure AI Search	Step 16; embedding output and AI Search index must exist.
18	Spend classification runs on cleansed spend lines.	Spend must be classified into L1/L2 taxonomy before category-level analysis is reliable.	Use tiered routing: decision cache first, similarity retrieval second, LLM only for uncertain lines, human review for low-confidence/high-spend lines.	Fabric notebooks, Azure OpenAI, vector index/cache, Human Review Queue	Step 8; cleansed spend lines in Silver must be available. L1/L2 taxonomy and taxonomy definitions must also be available.
19	Classification confidence is calculated.	Developers need a rule for auto-accept versus review.	Store classification_confidence, retrieval_score, top_candidate_gap, llm_reason, review_status, and final_category. Route records below threshold or high spend to review.	Fabric tables, review queue table, Cosmos/Fabric operational table	Step 18; classification output must be generated.
20	Human-reviewed classifications update the cache.	Every correction should reduce future LLM calls and improve consistency.	When a reviewer confirms or corrects a classification, write the decision to the classification cache keyed by supplier/item /product/category features.	Classification cache table, Fabric Gold/Silver, optional Cosmos operational table	Step 19; review queue and reviewer decisions must be available.
21	Supplier resolution runs after spend classification.	Supplier matching is more accurate when enriched with product and category context.	Build supplier profiles from raw supplier names, products sold, L1/L2 categories, supplier type, transaction history, and spend values.	Fabric notebooks, embeddings, clustering logic	Step 18 required; Step 19 required where confidence-based routing/review is used.
22	Supplier names are harmonized.	Different supplier name variants must roll up to one canonical supplier.	Normalize names, remove legal suffixes, embed supplier profiles, cluster similar suppliers, and use LLM verification for uncertain pairs or low-cohesion clusters.	Fabric Spark, Azure OpenAI embeddings, DBSCAN /HDBSCAN, LLM verification	Step 21; supplier profiles and resolution candidates must be generated.
23	Supplier parent-child mapping is created.	Parent-level spend is required for consolidation and strategic sourcing questions.	Use business registry data and Bing grounding to identify ultimate parent. Store evidence URL, source date, source quality, agreement score, and confidence.	Bing Grounded Search, business registry feed, Fabric supplier lookup table	Step 22; canonical supplier names must be available.
24	Supplier and parent mapping results are stored.	AskLab and Insights Lab need a reusable lookup table.	Create a supplier lookup table with raw_supplier_name, canonical_supplier_name, parent_supplier_name, cluster_id, confidence, parent_source_url, parent_source_date, review_status.	Fabric Gold Supplier Lookup table	Step 22 and Step 23; harmonized and parent mapping outputs must be approved.
25	Curated outputs are created for apps and agents.	Application and agents should not query raw tables directly.	Publish app-ready tables/views for PO facts, invoice facts, contract metadata, supplier lookup, taxonomy store, operational views, and supplier hierarchy.	Fabric Gold, Fabric Warehouse/Lakehouse SQL endpoint, curated views	Step 10, Step 15, Step 19, Step 20 where applicable, and Step 24; Gold data and approved lookup outputs must exist.
26	Curated operational views are synced to Cosmos where needed.	Cosmos supports low-latency operational app behavior, not authoritative procurement data.	Sync selected app-serving snapshots, saved questions, UI state, user feedback, and query history. Do not treat Cosmos as the primary source of procurement truth.	Azure Cosmos DB	Step 25; app-ready curated views must be published.
27	User opens PCC AskLab UI.	AskLab is the business front door for procurement questions.	Serve React web UI through Azure Container Apps or Static Web Apps depending on deployment decision.	PCC Front Door, ReactJS, Azure Container Apps	UI deployment must be complete; Step 63 for deployed environment.

28	User authenticates with Entra ID.	The system must know who the user is.	Implement Entra ID login. UI receives token /claims and passes token to backend.	Microsoft Entra ID, MSAL	Step 27; Entra app registration and authentication configuration must exist.
29	Backend validates authorization.	Authentication alone is not enough; backend must enforce what the user can do.	FastAPI middleware validates JWT, issuer, audience, expiry, scopes, app roles, and claims. Create an authorization policy component.	Python FastAPI, Entra ID JWT validation, Authorization Policy	Step 28; token/claims must be issued by Entra ID.
30	Backend creates user context.	Agents and tools need trusted user context for audit and authorization.	Build a context object containing user_id, roles, scopes, session_id, conversation_id, query_id, allowed_operations. Pass it downstream.	Backend API, Cosmos DB, App Insights	Step 29; token validation and authorization policy must pass.
31	AskLab query enters the hosted agent runtime.	The authorized user query must enter the AI orchestration layer before planning, clarification, routing, tool selection, or model reasoning.	Backend sends query text, session ID, conversation ID, correlation ID, and authorized user context to Azure AI Foundry Hosted Agent Runtime. Do not directly invoke the LLM here unless a runtime step requires it.	Backend API, Azure AI Foundry Hosted Agent Runtime, APIM/API route if used for runtime access	Step 30; authorized user context must exist. Hosted runtime endpoint must be deployed.
32	Approved model access is available to the runtime.	Runtime and agents need controlled model access, but model calls happen only when reasoning or generation is required.	Configure APIM / AI Gateway with approved Azure OpenAI deployments, rate limits, logging, throttling, request policies, and cost controls. Runtime calls the gateway only when needed.	APIM / AI Gateway, Azure OpenAI, Azure AI Foundry	Approved model routes must be configured before runtime steps requiring model calls.
33	Hosted Agent Runtime loads configured tools and context.	Agents cannot answer only from the user question; they need governed access to curated procurement data, ontology, session context, and contract evidence.	Configure the runtime with approved tool contracts for Fabric Gold, Fabric IQ /Ontology, Cosmos context, Supplier Lookup, Contract Metadata, and Azure AI Search. Validate connectivity and authorization before execution.	Azure AI Foundry Hosted Agents, Agent Tool Layer, Fabric Gold, Fabric IQ, Cosmos DB, Azure AI Search, Supplier Lookup, Contract Metadata Store	Step 31 and Step 32; Step 25 curated outputs ready; Step 26 Cosmos operational context ready where used; Step 17 Azure AI Search index ready; Step 11 Fabric IQ ready.
34	Understand Request step parses the query.	Plain English must be converted into structured intent before orchestration.	Extract intent, supplier mention, category, region, time window, contract need, and requested grain where possible. Use model reasoning only if rules/templates are insufficient.	Azure AI Foundry agent logic, Azure OpenAI via APIM / AI Gateway when required	Step 33; query, user context, session context, and tool/data availability must be loaded.
35	Clarification Gate checks completeness.	The system should not guess when required details are missing or ambiguous.	If supplier, category, time window, region, or contract intent is ambiguous or missing, return a targeted clarification question before tool execution. Use LLM only to phrase or reason over clarification when needed.	Azure AI Foundry agent logic, Backend response, Azure OpenAI via gateway if required	Step 34; structured request fields must be extracted.
36	Orchestrator Agent plans the execution.	A query may require one agent, multiple agents, or a staged execution path.	Classify query as spend, supplier, contract, or combined. Select specialist agents and tools. Define execution order, retry strategy, and fallback path. Use LLM planning only when deterministic routing is insufficient.	Orchestrator Agent in Azure AI Foundry, Agent Tool Layer, APIM / AI Gateway for model reasoning if required	Step 35; request must be complete or clarified. Agent catalogue and tool contracts must be available from Step 33.
37	Grain selection is decided.	Answers must not mix entity-level and parent-level supplier results.	Determine whether the query should run at raw entity, canonical supplier, or parent supplier grain. Carry this grain flag through tools and join workspace.	Orchestrator Agent, Join Workspace logic, Supplier Lookup	Step 36; orchestrator must classify query and execution intent. Supplier grain rules must exist in Supplier Lookup / curated model.
38	Tool authorization is enforced.	Agents must not have unrestricted access to tools or data.	Implement per-agent/per-tool authorization. Deny by default. Validate user context, role, scope, data access, and allowed operation before any tool call.	ToolAuth component, Managed Identity, RBAC, APIM policies	Step 30; user context must exist. Step 33; tool contracts must be loaded.

39	Query is routed to Spend Agent if spend-related.	Spend questions require curated PO, invoice, supplier, category, and time data.	Spend Agent calls authorized PO and Invoice tools. Apply supplier, category, region, and time filters using curated Gold data and ontology relationships. Use LLM only for interpretation or answer drafting if needed.	Spend Agent, PO Tool, Invoice Tool, Fabric Gold, Fabric IQ/Ontology, Azure OpenAI via gateway if required	Step 36, Step 37, Step 38; Step 25 curated PO/Invoice/Taxonomy outputs ready; Step 11 Fabric IQ ready.
40	Query is routed to Supplier Agent if supplier-related.	Supplier questions require canonical supplier, parent mapping, hierarchy, and relationship context.	Supplier Agent resolves typed supplier names using Live Supplier RAG, then calls authorized Supplier Lookup and Ontology tools. Use Bing grounding or LLM only for ambiguous supplier, parent, rebrand, or ownership cases.	Supplier Agent, Live Supplier RAG, Supplier Lookup Tool, Fabric IQ /Ontology, Bing Grounded Search, Azure OpenAI via gateway if required	Step 36, Step 37, Step 38; Step 24 supplier/parent mapping ready; Step 25 curated supplier outputs ready; Step 11 Fabric IQ ready.
41	Query is routed to Contract Agent if contract-related.	Contract questions require contract metadata and approved redacted contract evidence.	Contract Agent calls authorized Contract Metadata and Contract Search tools. It only searches approved redacted chunks. Use LLM only to interpret retrieved evidence or compose a supported answer.	Contract Agent, Contract Metadata Tool, Azure AI Search, Contract Metadata Store, Azure OpenAI via gateway if required	Step 36, Step 37, Step 38; Step 15 contract metadata ready; Step 17 Azure AI Search index ready.
42	Multi-agent questions invoke multiple agents.	Some questions require spend, supplier, contract, ontology, and session context together.	Orchestrator runs agents in parallel or staged mode depending on dependency. Example: resolve supplier first, then retrieve spend, invoices, contract coverage, and related context.	Orchestrator Agent, Spend Agent, Supplier Agent, Contract Agent, Fabric Gold, Fabric IQ, Cosmos DB, Azure AI Search	Step 36, Step 37, Step 38; relevant dependencies for Spend, Supplier, and Contract Agents must be ready.
43	Agent tools query governed curated stores.	Agents should not query raw or ungoverned data, and each tool must access only approved sources.	Tools call approved APIs or SQL/search endpoints over Fabric Gold, Fabric IQ, Azure AI Search, Cosmos context, Supplier Lookup, and Contract Metadata as needed. Deterministic retrieval should not invoke LLM.	Backend API/APIM, Fabric SQL endpoint, Fabric IQ /Ontology, Azure AI Search, Cosmos DB, Supplier Lookup, Contract Metadata Store	Step 38; authorization must pass. Step 25, Step 26, Step 11, and Step 17 must be ready depending on tool used.
44	AI Search query safety is applied.	Search queries can be abused if raw user input is passed directly.	Build search queries from templates. Use field allowlists, filter allowlists, escaped inputs, and rejected-query logging. Avoid raw string concatenation.	Azure AI Search, safe search query builder service	Step 17 and Step 38; AI Search index and authorized search tool must exist.
45	Contract search returns evidence chunks.	Contract answers need traceable evidence.	Return chunk text, contract ID, source file, clause/page, confidence, security metadata, and retrieval score.	Azure AI Search	Step 44; safe validated search request must execute successfully.
46	Tool outputs are normalized into canonical rows.	Different source outputs must align before being combined.	Return rows keyed by canonical_supplier_id and include selected grain, source name, filters used, timestamp, and confidence metadata.	Agent Tool Layer, Join Workspace	Step 39, Step 40, Step 41, or Step 42; tool outputs must be returned.
47	Neutral Join Workspace combines results.	No single source should dictate the combined answer.	Join PO, invoice, supplier, and contract rows using canonical_supplier_id. Use JOIN /INTERSECT depending on the question.	Join Workspace service /module	Step 46; source outputs must be normalized by canonical supplier ID.
48	Grain Check validates consistency.	Parent-level and entity-level results must not be mixed.	Compare returned row grain against selected grain flag. If inconsistent, fail validation and ask orchestrator to repair or re-run.	Join Workspace, Checker	Step 37 and Step 47; selected grain and joined result must exist.
49	Checker validates the evidence package.	Prevent unsupported, incomplete, or inconsistent answers.	Validate grounding, completeness, consistency, source quality, confidence, and whether all required parts of the query were answered. Use LLM only if judgement over evidence quality or completeness is required.	Checker component in Azure AI Foundry runtime, Azure OpenAI via gateway if required	Step 48; grain-consistent evidence package must be available.

50	Repair loop handles execution issues.	The system should retry intelligently instead of returning weak answers.	If a tool fails, evidence is weak, or grain is inconsistent, return to Orchestrator for retry, narrower scope, alternate tool path, or qualified response.	Repair/Replanning loop, Orchestrator Agent	Step 49; checker must identify execution, evidence, or grain issue.
51	Fallback handles missing or low-quality data.	The system must not hallucinate when data is unavailable or incomplete.	Return a qualified answer such as “I cannot confirm this from available data” and explain what data is missing or unsupported.	Fallback policy, Response Composer	Step 49 or Step 50; checker /repair must confirm answer cannot be fully supported.
52	Response Composer builds the final answer.	MVP AskLab response must be text-only and business-readable.	Generate a concise answer with evidence, caveats, assumptions, limitations, and source references where approved. Use LLM here for final answer synthesis when needed.	Response Composer, Azure OpenAI via APIM / AI Gateway	Step 49 if validated, or Step 51 if fallback is needed.
53	Sensitive Field Filter sanitizes the answer.	Prevent leaking PII, signatories, restricted metadata, or unapproved contract details.	Apply allowlist of fields that may be surfaced. Suppress, generalize, or redact restricted fields before returning response.	Sensitive field filter, policy config	Step 52; draft response must be generated.
54	Final response returns to Backend API.	Backend controls the response to UI and logs response metadata.	Return sanitized response, response ID, confidence/caveat metadata, and trace ID.	Backend API	Step 53; sanitized response must be ready.
55	UI displays answer to the user.	User receives the AskLab text-only response.	Render final answer, caveats, feedback buttons, and optional source/evidence section if approved.	React AskLab UI	Step 54; backend must return response to UI.
56	Chat history is stored.	Users can revisit previous conversations.	Store query, response metadata, timestamp, conversation ID, retention metadata, and trace ID. Avoid storing restricted payloads unnecessarily.	Cosmos DB	Step 54; query/response metadata must be available.
57	User feedback is captured.	Feedback supports UAT, quality improvement, and future tuning.	Capture helpful/not helpful and optional comment. Store with query ID, response ID, and conversation ID.	Cosmos DB Feedback Store	Step 55; user must see answer and submit feedback.
58	Logs and traces are captured.	Developers need traceability for debugging, audit, observability, and quality review.	Log parsed request, route, selected agent, tools called, model calls where used, result status, fallback reason, latency, token usage, and errors. Avoid sensitive raw content where not needed.	App Insights, Log Analytics, OpenTelemetry	Runs across Step 27 to Step 57; App Insights / Log Analytics must be configured.
59	Monitoring tracks health and performance.	Operations must detect failures, latency issues, tool errors, model usage spikes, and rejected requests.	Build dashboards for API errors, agent failures, tool failures, AI Search errors, model-call volume, latency, rejected queries, and pipeline failures.	Azure Monitor, App Insights, Log Analytics	Step 58; telemetry must be generated.
60	Key Vault stores secrets and keys.	Secrets must not be embedded in code or configuration files.	Use managed identity wherever possible. Store secrets, connection strings, certificates, and API keys in Key Vault.	Azure Key Vault, Managed Identity	Required before Step 6, Step 27, Step 31, Step 43, and Step 63.
61	Defender monitors security posture.	Platform security must be continuously checked.	Enable Defender for Cloud recommendations for storage, container apps, registry, key vault, APIs, databases, and relevant AI /search resources.	Microsoft Defender for Cloud	Required before production hardening; should be enabled before Step 63.
62	Azure Policy enforces governance.	Environments should remain compliant by design.	Apply policies for diagnostics, public access, encryption, allowed regions, private endpoints where required, and tagging.	Azure Policy	Required before environment promotion; should be configured before Step 63.

63	DevOps pipelines deploy the solution.	Build and deployment must be repeatable and auditable.	Use GitHub Actions for build, test, scan, container image push, IaC deployment, and environment promotion.	GitHub, GitHub Actions, Container Registry, IaC	Code, IaC, configuration, Key Vault, container registry, and environment setup must be ready.
64	Review outcomes improve the platform.	Human review should not be wasted.	Feed corrected classification, supplier mapping, contract linkage, and user feedback into cache and curated tables where approved.	Review Queue, Fabric Gold, Supplier Lookup, Taxonomy Store, Cosmos feedback	Step 20, Step 57, and Step 58; corrections, feedback, and traces must be available.
65	The solution is governed end-to-end.	PCC handles sensitive procurement and contract data.	Maintain evidence for data classification, access control, retention, logging, validation, quality review, UAT, deployment, risk closure, and residual-risk acceptance.	SharePoint / Confluence, JIRA, GitHub evidence, monitoring exports	Depends on all previous delivery, evidence, monitoring, security, and handover steps.